

Réseaux pour le cloud

Jour 4 — Le réseau **DANS** le cloud : VPC, security groups, interconnexion

Tout ce que vous avez appris cette semaine converge ici — et devient
le plan du TP2

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Décrire l'**anatomie d'un VPC/VNet** : CIDR, sous-réseaux publics/privés, tables de routage, **Internet Gateway**, **NAT Gateway**.
- Expliquer ce qui **change** (et ce qui ne change pas) par rapport au réseau physique.
- Distinguer **security groups** (stateful) et **NACL** (stateless) et prédire qui bloque quoi.
- Comparer les modes d'**interconnexion** : peering, VPN site-à-site managé, Direct Connect / ExpressRoute.
- **Concevoir sur papier un plan d'adressage complet** – celui de StockLine 3-tiers multi-AZ, que vous construirez au TP2.

Vocabulaire du jour = vocabulaire d'AWS. La console, elle, attend le bloc CL-AWS1.

Plan de la journée

0. **Échauffement subnetting** (10 min – les 3 derniers, promis*).
1. **Anatomie d'un VPC** – CIDR, sous-réseaux, routage, IGW, NAT GW.
2. **Ce qui change** par rapport au réseau physique.
3. **Security groups vs NACL** – stateful vs stateless, en pratique.
4. **Interconnexion** – peering, VPN site-à-site, Direct Connect/ExpressRoute.
5.  **Atelier fil rouge** – le plan d'adressage StockLine 3-tiers multi-AZ.
6. **Préparation du TP1** (qui commence lundi).
7. Récap + quiz.

*Non. Vous en referez au TP1, au TP2, et en entretien.



Échauffement – 3 calculs orientés cloud (10 min)

1. Un VPC en `10.42.0.0/16` : combien d'adresses au total ?
2. Découpez-y les deux premiers sous-réseaux `/24`. Réseau, plage, broadcast ?
3. `10.42.10.0/24` chez AWS : sachant qu'AWS réserve **5 adresses** par sous-réseau, combien reste-t-il d'adresses utilisables ?

(Réponse à la 3 dans dix minutes si personne ne la trouve – indice : $2^8 - 5$.)

1. Anatomie d'un VPC

Votre datacenter, défini en cinq objets

Le VPC : un réseau privé virtuel et isolé

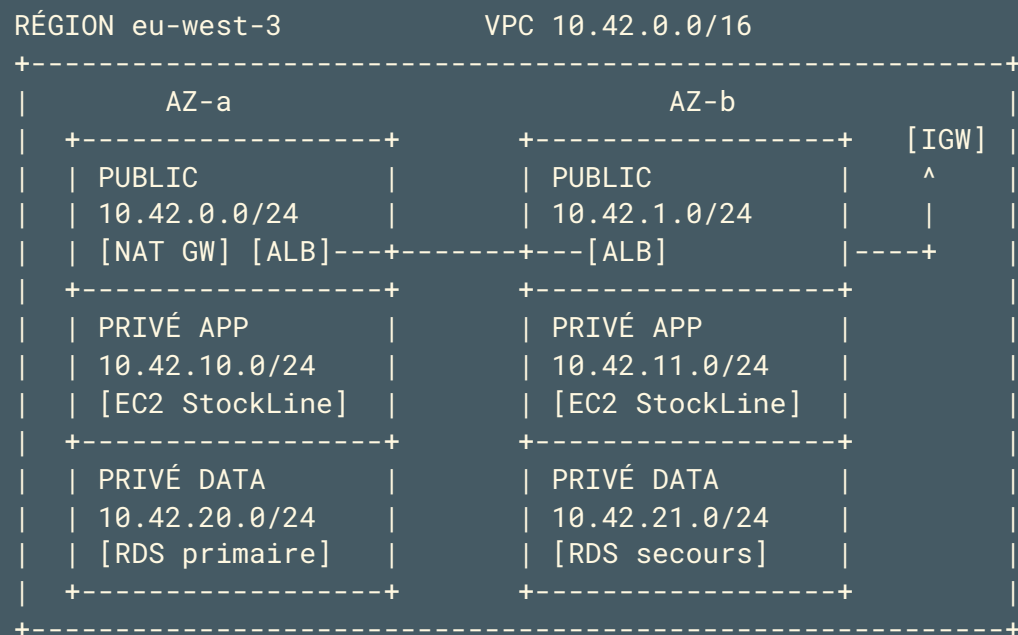
Un **VPC** (Virtual Private Cloud — **VNet** chez Azure) est **votre** réseau privé, isolé, dans une région du fournisseur :

- vous choisissez son **CIDR** (ex. `10.42.0.0/16`) — plage **privée** RFC 1918, entre /16 et /28 chez AWS ;
- il s'étend sur **une région**, et couvre **plusieurs zones de disponibilité** (AZ = datacenters distincts de la même région) ;
- **rien n'y entre ni n'en sort** sans un objet explicite (gateway, peering...) : isolation par défaut ;
- tout ce qu'il contient (instances, bases...) vit **dans un sous-réseau** du VPC.

Choix du CIDR = décision structurante :

- assez **large** pour des années de croissance (un /16 = 65 536 adresses) ;
- **sans chevauchement** avec vos autres réseaux (on ne peut ni peerer ni connecter deux plages qui se recouvrent — et on ne change pas un CIDR facilement après coup).

Vue d'ensemble : le schéma à savoir redessiner



Les sous-réseaux : votre découpage CIDR, appliqué

Un **sous-réseau** (subnet) est une tranche du CIDR du VPC, **dans une seule AZ** :

- c'est **exactement** le découpage de l'adresse : `10.42.0.0/16` → des `/24` ;
- multi-AZ = **dupliquer chaque rôle dans au moins 2 AZ** (d'où les paires 0/1, 10/11, 20/21 du schéma) ;
- les tailles n'ont pas besoin d'être égales : du VLSM (exercice 1-2 !) — petits sous-réseaux publics, grands sous-réseaux applicatifs.

⚠ Particularité AWS : **5 adresses réservées** par sous-réseau — réseau (`.0`), routeur VPC (`.1`), DNS (`.2`), réservée (`.3`), broadcast (`.255`). Un `/24` offre donc **251** adresses, pas 254.

(Et il n'y a **pas de broadcast** dans un VPC : l'adresse est réservée quand même.)

Un sous-réseau n'est ni public ni privé « par nature » :

c'est sa table de routage qui le dit → slide suivante.

Tables de routage : le mardi appliqué au cloud

Chaque sous-réseau est associé à **une table de routage** — les mêmes règles que `ip route`, écrites dans la console ou en Terraform :

Table "publique"	Table "privée"
10.42.0.0/16 -> local	10.42.0.0/16 -> local
0.0.0.0/0 -> igw-0abc12	0.0.0.0/0 -> nat-0def34

- `local` : tout le VPC est routé d'office entre ses sous-réseaux (non supprimable) ;
- `0.0.0.0/0` : la **route par défaut** — vous la connaissez ;
- **longest prefix match** : identique à mardi — une route `/16` vers un peering bat la route par défaut pour ce trafic-là.

Définitions officielles à retenir :

sous-réseau **public** = sa table route `0.0.0.0/0` vers l'**Internet Gateway** ;

sous-réseau **privé** = pas de route vers l'IGW (au mieux, vers une NAT GW).

Internet Gateway et NAT Gateway : qui sort, qui entre

Internet Gateway (IGW) — la porte sur Internet du VPC :

- un par VPC, hautement disponible, sans capacité à gérer ;
- fait le **NAT 1:1** entre l'IP privée d'une instance et son **IP publique** (l'instance ne voit jamais son IP publique — souvenez-vous du DNAT de mardi) ;
- trafic **entrant et sortant** — pour les sous-réseaux publics.

NAT Gateway (NAT GW) — la sortie discrète des sous-réseaux privés :

- un **SNAT managé** (mardi encore) : les instances privées **sortent** (apt, API, dépôts) mais **restent injoignables** depuis Internet ;
- vit **dans un sous-réseau public** (il lui faut l'IGW pour sortir) ;
- une par AZ en production (sinon : dépendance inter-AZ).

💰 **Coût** : la NAT GW est **facturée à l'heure + au Go** (~0,05 \$/h) — un des pièges de facturation classiques. Toujours dans le teardown des TP.

Le trajet d'un paquet : deux scénarios à dérouler

- 1) Client Internet -> StockLine :
Internet -> [IGW] -> table publique -> [ALB, subnet public]
-> cible 10.42.10.x (route "local") -> [EC2 app]
- 2) EC2 privée fait "apt update" :
[EC2 10.42.10.x] -> table privée : 0.0.0.0/0 -> [NAT GW]
-> table publique : 0.0.0.0/0 -> [IGW] -> Internet
(réponse : chemin inverse, la NAT GW retrouve la connexion)
- 3) EC2 app -> RDS 10.42.20.x :
route "local", ne quitte JAMAIS le VPC

Savoir dérouler ces trois trajets à l'oral = la moitié du quiz VPC de la certification AWS.

2. Ce qui change (et ce qui ne change pas)

par rapport au réseau physique

Ce qui ne change pas / ce qui change

Invariants — tout ce que vous avez appris reste vrai :

- CIDR, masques, plages privées, longest prefix match, NAT, DNS, TLS, stateful/stateless : **identiques au bit près**.

Ce qui change :

Réseau physique	Réseau cloud
câbler, racker, configurer des switchs	déclarer (console, API, Terraform)
couche 2 visible (ARP, broadcast, VLAN)	couche 2 simulée — pas de broadcast
pare-feu périmétrique central	filtrage au plus près de la ressource (SG)
une erreur = déplacement, recâblage	une erreur = recréer en 30 secondes
capacité achetée à l'avance	passerelles managées, facturées à l'usage

Deux conséquences pratiques : le sniffing type tcpdump ne voit **que votre trafic** (l'hyperviseur filtre) ; et comme tout est déclaratif, **tout est versionnable** — d'où l'IaC du bloc 11. Le réseau devient du code.

3. Security groups vs NACL

Le stateful/stateless d'hier, en production

Le security group : pare-feu stateful de la ressource

Un **security group (SG)** s'attache à une ressource (instance EC2, RDS, ALB...) :

- **stateful** : réponse automatiquement autorisée (le conntrack d'hier) ;
- règles **allow uniquement** — tout ce qui n'est pas autorisé est refusé ;
- toutes les règles sont **évaluées ensemble** (pas d'ordre, pas de numéros) ;
- défaut : **tout entrant refusé, tout sortant autorisé** ;
-  une règle peut citer **un autre SG comme source** — c'est la killer feature :

```
sg-alb : IN  80,443  depuis 0.0.0.0/0
sg-app : IN  8000    depuis sg-alb      <- "seul l'ALB me parle"
sg-db  : IN  5432    depuis sg-app      <- "seule l'app me parle"
```

Trois lignes = l'architecture 3-tiers sécurisée, **sans une seule IP en dur**.

L'Auto Scaling peut créer des instances : elles héritent du bon filtre.

La NACL : pare-feu stateless du sous-réseau

Une **NACL** (Network ACL) s'applique **au sous-réseau** entier :

- **stateless** : le trafic retour doit être **explicitement** autorisé (les ports éphémères d'hier : **1024-65535**) ;
- règles **allow ET deny**, évaluées **par numéro croissant, première correspondance gagne**, avec un *** deny** implicite à la fin ;
- la NACL par défaut autorise tout ; une NACL **custom** part de « tout refusé ».

```
#100 allow IN TCP 443 depuis 0.0.0.0/0
#110 deny IN TCP 443 depuis 198.51.100.0/24 <- jamais atteinte ! (après #100)
#120 allow OUT TCP 1024-65535 vers 0.0.0.0/0 <- le RETOUR du 443 entrant
* deny tout
```

Usage réel : la NACL sert surtout à **bannir** (deny d'une plage hostile), le SG à **autoriser** finement. Le quotidien, c'est 95 % de SG.

SG vs NACL : le tableau à connaître par cœur

	Security group	NACL
S'applique à	la ressource (ENI)	le sous-réseau
État	stateful	stateless
Types de règles	allow seulement	allow et deny
Évaluation	toutes les règles	par numéro , 1re qui matche
Trafic retour	automatique	règle explicite (ports éphémères)
Source possible	CIDR, autre SG	CIDR seulement
Défaut	IN refusé / OUT autorisé	défaut : tout ; custom : rien

Un paquet entrant traverse **la NACL du sous-réseau PUIS le SG** de la ressource : les deux doivent laisser passer. Au retour, la NACL revérifie (stateless), le SG non (stateful).

Symptôme ultra-classique : « la connexion s'établit puis les réponses se perdent » → **NACL sans règle sortante éphémère**. Exercice 4-2 : vous jouerez le pare-feu vous-mêmes.



Démo 4-1 — Décortiquer un vrai VPC

(lecture seule)

Fichier : `demons/04-cl-reseau/demo-4-1-decortiquer-un-vpc.md`

Visite guidée de la console AWS — **compte formateur, lecture seule, vous ne touchez à rien aujourd'hui** (la manipulation, c'est CL-AWS1) :

- le VPC par défaut : son CIDR, ses sous-réseaux par AZ ;
- un VPC « StockLine » pré-crée conforme au schéma du matin ;
- chaque objet du vocabulaire, **vu en vrai** : tables de routage (`0.0.0.0/0 → igw-...` / `→ nat-...`), IGW, NAT GW, SG avec référence croisée `sg-...`, NACL et ses numéros de règles.

Objectif : vendredi prochain, quand vous ouvrirez cette console en CL-AWS1, **vous aurez déjà tout le vocabulaire**. Il ne restera que les clics.

4. Interconnexion

Relier des VPC entre eux et au monde

VPC peering : relier deux VPC

Le **peering** connecte deux VPC en privé (même compte ou non, même région ou non) — le trafic ne passe jamais par Internet :

```
VPC prod 10.10.0.0/16 <===== pcx-abc123 =====> VPC dev 10.30.0.0/16  
table : 10.30.0.0/16 -> pcx-abc123      table : 10.10.0.0/16 -> pcx-abc123
```

Trois règles structurantes :

1. **CIDR sans chevauchement obligatoire** — deux VPC en `10.0.0.0/16` ne se peereront **jamais**. C'est pour ça qu'on planifie (atelier tout à l'heure : prod 10.10, staging 10.20, dev 10.30...).
2. **Routes à ajouter des deux côtés** — la panne « aller sans retour » de mardi, en version cloud.
3. **Pas transitif** : $A \leftrightarrow B$ et $B \leftrightarrow C$ n'impliquent pas $A \leftrightarrow C$. Au-delà de quelques VPC → **Transit Gateway** (hub central ; retenez le nom).

Vers le on-premise : VPN managé ou lien dédié

Relier le datacenter de l'entreprise au VPC — deux options :

VPN site-à-site managé (AWS Site-to-Site VPN) :

- le VPN d'hier, dont AWS gère un côté : Virtual Private Gateway côté VPC ↔ Customer Gateway (votre routeur) ; tunnels IPsec **redondés** ;
- rapide à monter (heures), coût modéré, mais débit limité (~1,25 Gb/s/tunnel) et latence Internet.

Direct Connect (AWS) / **ExpressRoute** (Azure) :

- une **liaison physique dédiée** entre votre réseau et le fournisseur ;
- débit garanti (1-100 Gb/s), latence stable, trafic hors Internet public ;
- délais en **semaines** et coût élevé → pour les besoins sérieux et durables.

Architecture courante : **Direct Connect en nominal + VPN en secours.**

Et dans tous les cas, la règle d'or : **CIDR on-premise et CIDR VPC ne doivent pas se chevaucher** — d'où le scénario 2 de l'exercice 4-1.



Exercice 4-1 — Plans d'adressage VPC : 3 scénarios (50 min)

Fichier : `exercices/04-cl-reseau/exercice-4-1-plans-adressage-vpc.md`

Trois entreprises, trois cahiers des charges, à vous de jouer l'architecte :

1. **PME e-commerce** : un VPC, 2 AZ, web public + app/db privés — le cas nominal.
2. **Groupe hybride** : un datacenter existant en `10.0.0.0/16` et `192.168.0.0/16`, VPN site-à-site prévu — choisir un CIDR **sans chevauchement** et le justifier.
3. **Multi-environnements** : prod (3 AZ) + staging + dev, peering entre eux — plan d'ensemble cohérent et extensible.

Livrable par scénario : CIDR du VPC justifié, tableau des sous-réseaux (nom, AZ, CIDR, plage, rôle), routes principales. **Tout calcul posé.**



Exercice 4-2 — SG vs NACL : qui bloque quoi ? (30 min)

Fichier : `exercices/04-cl-reseau/exercice-4-2-sg-vs-nacl.md`

On vous donne une configuration complète (SG + NACL sur l'architecture StockLine) et **huit paquets candidats** :

- pour chacun : passe / bloqué ? **par quoi** (SG ? NACL ? quelle règle) ? à l'aller ou au retour ?
- deux pannes à diagnostiquer, dont l'incontournable « le handshake passe, la réponse meurt » ;
- bonus : corriger la NACL fautive avec les bons numéros de règles.

C'est l'exercice qui distingue ceux qui ont **compris** stateful/stateless de ceux qui l'ont appris par cœur.

5. Atelier fil rouge

Le plan d'adressage StockLine 3-tiers multi-AZ

La commande : StockLine en production sur AWS

Cahier des charges (c'est **exactement** l'architecture du TP2, bloc 07) :

- l'API **StockLine** (FastAPI, port 8000) sur des instances EC2 en **Auto Scaling** (2 à 6 instances) ;
- un **ALB** public en HTTPS devant ;
- **RDS PostgreSQL Multi-AZ** (port 5432, primaire + secours) ;
- haute disponibilité : **2 AZ** (eu-west-3a, eu-west-3b) ;
- les instances privées doivent pouvoir sortir (mises à jour, dépôts) ;
- prévoir la croissance : un futur tier « batch/workers » et une 3e AZ doivent pouvoir s'ajouter **sans renuméroter**.

À produire en binôme, sur papier / tableau blanc (60 min) :

1. CIDR du VPC (justifié) ; 2. tableau complet des sous-réseaux ;
2. les deux tables de routage ; 4. les trois SG avec leurs références croisées ; 5. le schéma ASCII de l'ensemble.

Atelier — les questions qui fâchent (à traiter dans votre plan)

Votre plan doit répondre explicitement à ces questions :

- Pourquoi **ce** CIDR de VPC et pas un autre ? (chevauchements futurs ? peering avec un VPC dev un jour ?)
- Des `/24` partout, ou du VLSM ? Qu'est-ce que ça coûte de voir trop petit ? (indice : on ne renumérote pas un sous-réseau, on le détruit)
- Où vivent les **NAT Gateway**, et combien ? (une ? une par AZ ? argumentez coût vs panne d'AZ)
- RDS a-t-il besoin de sortir sur Internet ? Sa table de routage doit-elle contenir `0.0.0.0/0` ? (défaut fermé !)
- Qui a le droit de parler à qui, port par port ? (vos trois SG)
- Où reste-t-il de la place pour le tier « batch » et l'AZ-c ?

Restitution : deux binômes au tableau, les autres **challenger**.

Le corrigé de référence est dans `solutions/04-cl-reseau/atelier-fil-rouge-stockline.md`

— à ne pas ouvrir avant la restitution.

Le livrable attendu (structure, pas les réponses !)

```

1. VPC : 10.____.0.0/16    "parce que ..."

2. | Sous-réseau   | AZ | CIDR           | Utilisables | Rôle       |
   | public-a     | a  | 10._____.0/24  | 251          | ALB,NAT    |
   | ...          |    |                 |              |            |

3. Table publique : 0.0.0.0/0 -> ____
   Table privée   : 0.0.0.0/0 -> ____

4. sg-alb : IN ___ depuis ___
   sg-app  : IN ___ depuis ___
   sg-db   : IN ___ depuis ___

5. Schéma ASCII complet (modèle : slide "vue d'ensemble" du matin)

```

6. Préparation du TP1

Lundi, tout ce bloc devient une note

CL-TP1 : ce qui vous attend lundi et mardi

TP noté sur 2 jours (bloc 05) : déployer StockLine sur une VM Linux, de bout en bout. La part réseau de la grille d'évaluation :

- **plan d'adressage documenté** de la VM (adresse, masque en CIDR, passerelle, DNS — et vérifié avec `ip a` / `ip route` / `resolvectl`) ;
- **nginx en reverse proxy** devant l'API (les ports de mercredi : 80/443 publics, 8000 local) ;
- **pare-feu ufw** : politique par défaut fermée, règles justifiées (exercice 3-2 = votre brouillon) ;
- diagnostic : prouver avec `ss`, `curl`, `dig` que chaque étage répond.

Ce week-end (45 min max) : refaites **5 calculs** de l'exercice 1-2 sans regarder la méthode, relisez la cheatsheet (`cheatsheet/04-cl-reseau.md`), vérifiez que votre VM démarre et que vous avez la main en SSH.

Récap des points clés du jour

- **VPC** = votre CIDR privé dans une région, découpé en **sous-réseaux** mono-AZ ; AWS réserve **5 adresses** par sous-réseau (251 dans un /24).
- Public vs privé = **une table de routage** : `0.0.0.0/0 → IGW` (public) ou `→ NAT GW` (privé qui doit sortir). Rien d'autre.
- **IGW** = porte d'entrée/sortie, NAT 1:1 ; **NAT GW** = SNAT managé, vit en sous-réseau public, 💰 facturée.
- **SG** : stateful, allow only, source = autre SG → le 3-tiers en 3 lignes.
NACL : stateless, numéros, deny possible, penser aux **ports éphémères**.
- **Peering** : CIDR disjoints, routes des deux côtés, non transitif ;
VPN site-à-site = vite monté ; **Direct Connect/ExpressRoute** = lien physique dédié.
- Un plan d'adressage se conçoit **avant** de cliquer — le vôtre servira au TP2 tel quel.



Quiz — questions 1 et 2

Question 1

Un VPC peut-il s'étendre sur plusieurs régions ? Sur plusieurs zones de disponibilité ? Et un sous-réseau ?

Question 2

Qu'est-ce qui fait officiellement d'un sous-réseau un sous-réseau **public** ?
(La réponse tient en une ligne de table de routage.)



Quiz — questions 3 et 4

Question 3

Une instance en sous-réseau privé doit télécharger ses mises à jour, mais ne doit jamais être joignable depuis Internet. Quel composant utilisez-vous, dans quel sous-réseau vit-il, et quel mécanisme de marni implémente-t-il ?

Question 4

Dans un `/24` AWS, combien d'adresses sont utilisables par vos instances, et pourquoi ce nombre diffère-t-il du calcul classique $2^8 - 2$?



Quiz — questions 5 et 6

Question 5

Security group : stateful ou stateless ? Quelles conséquences pour le trafic de réponse ? Et quel est son comportement par défaut en entrée et en sortie ?

Question 6

Comment une NACL évalue-t-elle ses règles ? Que se passe-t-il si la règle #100 autorise ce que la règle #110 refuse ?



Quiz — questions 7 et 8

Question 7

Plutôt que d'autoriser le port 5432 depuis `10.42.10.0/24`, on recommande d'autoriser le port 5432 depuis `sg-app`. Quel est l'avantage décisif de cette écriture quand l'Auto Scaling crée et détruit des instances ?

Question 8

Deux VPC en `10.0.0.0/16` chacun peuvent-ils être peerés ? Pourquoi ?
Quelle décision prise **aujourd'hui** dans un plan d'adressage évite ce problème dans deux ans ?



Quiz — questions 9 et 10

Question 9

Votre client exige un débit garanti de 10 Gb/s et une latence stable entre son datacenter et son VPC, hors Internet public. VPN site-à-site ou Direct Connect ? Justifiez, et citez l'architecture de secours courante.

Question 10

Dans votre plan StockLine : dans quels sous-réseaux placez-vous RDS, et sa table de routage contient-elle `0.0.0.0/0` ? Justifiez.

Bravo — le bloc réseau est terminé !

Lundi : CL-TP1 — votre plan d'adressage devient une note

Vous savez calculer un sous-réseau, suivre un paquet, lire un certificat,
écrire un pare-feu et dessiner un VPC. Lundi, vous le prouvez sur une VM.

Et dans trois semaines, au TP2, le plan StockLine d'aujourd'hui deviendra
une vraie infrastructure multi-AZ. Gardez précieusement vos